

Week 2 - Lab 2: Applied STL Challenge

Thursday Lab | Independent Practice: Game Score Analyser

Module: DATA: Data Structures and AI Algorithms

Week: 2

Duration: 2 hours

How this lab is different: This lab gives less step-by-step code. Use Wednesday's lab and the Week 2 lecture as your reference. You should decide how to combine the STL operations yourself.

Learning outcomes

- Apply vector operations without a complete worked solution.
- Traverse a vector using iterators.
- Combine `std::sort`, `std::find`, `std::reverse` and `std::for_each`.
- Use a lambda to perform an action on every element.
- Recognise where the container, iterators and algorithm appear in your own program.

Scenario

You are building a small score-analysis tool for a game. The game stores recent player scores and needs to display, sort, search and process them. Your task is to build the tool using STL rather than writing your own sorting or searching algorithms.

Rules

- Use `std::vector<int>` as the main container.
- Use at least three STL algorithms.
- Use an iterator-based loop at least once.
- Use `std::for_each` with a lambda at least once.
- Do not manually implement a sorting algorithm.

Starting data

```
std::vector<int> scores
{
    4200, 1800, 7600, 3500, 5900,
    2400, 8100, 4600, 3500, 6700
};
```

Task 1 - Inspect the data (about 15 minutes)

1. Display the original scores.
2. Display how many scores are stored.
3. Display the first score.
4. Display whether the vector is empty.

Task 2 - Traverse using iterators (about 15 minutes)

5. Create an iterator using `scores.begin()`.
6. Print the first value using `*it`.
7. Move to the next value using `++it` and print it.
8. Write a full iterator loop from `begin()` to `end()` that prints every score.

Self-check: Your loop condition should stop when the iterator reaches `scores.end()`. Do not dereference `end()`.

Task 3 - Build the leaderboard (about 20 minutes)

9. Sort the scores into ascending order with `std::sort`.
10. Print the sorted scores.
11. Use `std::reverse` to change the order so the highest score appears first.
12. Print the leaderboard again.

Q3A. Why is using `std::sort` preferable here to writing your own sorting routine for this task? Answer with one or two sentences in a comment.

Task 4 - Search the leaderboard (about 20 minutes)

Ask the user to enter a score to search for. Then use `std::find` to determine whether that score exists.

- If found, print: Score found.
- If not found, print: Score not found.
- Test once with a score that exists (for example 5900).
- Test once with a score that does not exist.

Hint: `std::find` returns an iterator. Compare the returned iterator with `scores.end()` to decide whether the value was found.

Task 5 - Process every score with a lambda (about 20 minutes)

13. Use `std::for_each` with a lambda to print every score.
14. Change the lambda so it prints each score followed by the text " points".
15. Use another `std::for_each` call with an `int&` parameter to add a 250-point bonus to every stored score.
16. Print the vector after the bonus and confirm that the stored values changed.

Task 6 - Build one complete report (about 20 minutes)

Your program should now print a short report in this order:

17. Original scores.
18. Number of scores and the first score.
19. Scores in highest-to-lowest order.
20. Result of the user's search.

21. Scores after the 250-point bonus.

Design requirement: Keep the output easy to read. Use headings such as "Original Scores", "Leaderboard", "Search Result" and "After Bonus".

Stretch challenge - choose ONE if you finish early

Option A - Genericity: use the same idea with strings

- Create `std::vector<std::string>` `playerNames` with at least five names.
- Sort the names alphabetically.
- Search for one name using `std::find`.
- Use `std::for_each` to display all names.

Option B - Associative container: simple player lookup

- Create a `std::map<std::string, int>` mapping player names to scores.
- Add at least four players.
- Ask for a player name and display that player's score.
- What is the 'key' and what is the 'value' in your map?

Hint for Option B: You will need `#include <map>`. The lecture example used a player name as the key and a score as the value.

Final self-check

Check	Done?
My program uses a vector.	☐
I used <code>begin()</code> and <code>end()</code> .	☐
I traversed with an iterator.	☐
I used <code>sort()</code> .	☐
I used <code>find()</code> .	☐
I used <code>reverse()</code> .	☐
I used <code>for_each()</code> with a lambda.	☐
I tested both found and not-found searches.	☐
My program compiles without errors.	☐

Short reflection

- In your program, identify one container.
- Identify one pair of iterators.

- Identify two algorithms.
- In one sentence, explain: Container stores data -> Iterators define the range -> Algorithm does the work.

End of lab: Your finished program should be your own work. Use the lecture and Lab 1 for guidance, but try to solve each task before asking for the exact code.